СТРУКТУРА И СТАНДАРТНАЯ БИБЛИОТЕКА

# Kotlin: презентация №2

Команда 1: Шутова Ю., Матвейчик Е., Корся Е.

# Выходные данные и возврат значений

## Возврат результата из функции

- **Оператор return:** завершает вызов и передаёт вычисленное значение обратно
- **Тип результата:** указывается после круглых скобок через двоеточие (: Int)
- **Функции-выражения:** запись через '=' в одну строку без '{}' и return

## Специальные случаи возврата

- **Несколько значений:** классы Pair и Triple с удобной распаковкой переменных
- **Тип Unit:** аналог void — возвращается, когда функция только выполняет действие

p2_01_return_values.kt

```kotlin
fun square(number: Int): Int {
    return number * number
}

// Функция-выражение (Single-expression)
fun cube(n: Int) = n * n * n

fun minMax(a: Int, b: Int): Pair<Int, Int> =
    if (a < b) Pair(a, b) else Pair(b, a)

fun main() {
    println("Квадрат 5: ${square(5)}, Куб 3: $
{cube(3)}")
    val (minV, maxV) = minMax(42, 17)
    println("Мин: $minV, Макс: $maxV")
}
```

# Условные конструкции: if-else и when

p2_02_conditionals.kt

```kotlin
fun evaluateScore(score: Int): String {
    val status = if (score >= 60) "Зачёт" else
"Пересдача"
    val grade = when (score) {
        in 90..100 -> "Отлично (A)"
        in 75..89  -> "Хорошо (B)"
        in 60..74  -> "Удовлетворительно (C)"
        else       -> "Неудовлетворительно"
    }
    return "$status — $grade"
}

fun inspect(obj: Any): String = when (obj) {
    is Int    -> "Число: ${obj * 2}"
    is String -> "Строка длиной ${obj.length}"
    else      -> "Неизвестный тип"
}
```

## if-else как выражение

- **Возвращает результат:** значение ветки можно сразу присвоить в переменную
- **Без тернарного оператора:** запись if (a > b) a else b полностью заменяет a ? b : c

## Конструкция when (замена switch)

- **Гибкие условия:** проверка констант, диапазонов (in 90..100) и списков значений
- **Проверка типов (is):** в сочетании со Smart Cast открывает свойства объекта без каста
- **Безопасность:** не нужен break; компилятор проверяет полноту всех веток

КОНСТРУКЦИИ ПОТОКА УПРАВЛЕНИЯ

# Циклы: for, while, диапазоны и метки

## Цикл for и диапазоны (Ranges)

- **Принцип for-each:** перебирает диапазоны (1..10), массивы и любые коллекции
- **Гибкий шаг:** операторы until (без границы), downTo (назад) и step (шаг)
- **Доступ к индексам:** метод withIndex() возвращает пару (index, element)

## Циклы while / do-while и метки

- **Пред- и постусловие:** while проверяет условие до шага, do-while — после
- **Именованные метки:** break@outer и continue@outer для управления вложенными циклами

p2_03_loops.kt

```kotlin
fun main() {
    for (i in 10 downTo 2 step 2) {
        print("$i ") // 10 8 6 4 2
    }

    val langs = listOf("Kotlin", "Java", "Rust")
    for ((idx, lang) in langs.withIndex()) {
        println("[$idx] -> $lang")
    }

    var sum = 0
    outer for (r in 1..3) {
        for (c in 1..3) {
            sum += r * c
            if (sum > 10) break@outer
        }
    }
}
```

# Итераторы, последовательности и блоки

p2_04_iterators_blocks.kt

```kotlin
class Countdown(val start: Int) : Iterable<Int> {
    override fun iterator() = object : Iterator<Int> {
        var cur = start
        override fun hasNext() = cur > 0
        override fun next() = cur--
    }
}

fun main() {
    val evens = (1..1000).asSequence()
        .filter { it % 2 == 0 }
        .map { it * it }
        .take(3).toList()
    println("Квадраты: $evens")

    val area = run { val w = 12; val h = 5; w * h }
}
```

## Итераторы и ленивые Sequence

- **Контракт Iterator<T>:** методы hasNext() и next() позволяют обходить любой класс
- **Ленивые Sequence:** вычисляют элементы по одному без создания промежуточных списков
- **Экономия ресурсов:** take(3) останавливает конвейер сразу после 3 совпадений

## Блоки кода и изоляция видимости

- **Локальная область:** переменные внутри блока {} удаляются сразу при выходе
- **Блок run { ... }:** выполняет группу команд и возвращает результат последней строки

ОБРАБОТКА ОШИБОК

# Исключения: try-catch-finally и Result

## Исключения в Kotlin

- **try-catch как выражение:** возвращает значение из блока try или резерв из catch
- **Блок finally:** выполняется гарантированно при любом сценарии завершения
- **Без Checked Exceptions:** нет обязательных throws в сигнатурах, код остаётся чистым

## Функциональный подход runCatching

- **Обёртка Result<T>:** перехватывает ошибку и хранит либо успех, либо исключение
- **Безопасное извлечение:** методы getOrNull(), getOrElse {}, onFailure {}

p2_05_exceptions.kt

```kotlin
fun parsePort(input: String): Int =
    try {
        val port = input.toInt()
        if (port !in 1..65535) throw
IllegalArgumentException()
        port
    } catch (e: NumberFormatException) {
        8080 // Резервный порт по умолчанию
    } finally {
        println("Проверка '$input' завершена")
    }

fun main() {
    println(parsePort("9090")) // 9090
    val res = runCatching { "123x".toInt() }
    println(res.getOrElse { -1 }) // -1
}
```

# Встроенные средства отладки и контракты

p2_06_assertions_contracts.kt

```kotlin
class BankAccount(initial: Double) {
    var balance: Double = initial
        private set
    init {
        require(initial >= 0) { "Баланс < 0" }
    }
    fun withdraw(amount: Double) {
        require(amount > 0) { "Сумма <= 0" }
        check(balance >= amount) { "Мало средств" }
        balance -= amount
        assert(balance >= 0) { "Ошибка инварианта" }
    }
}
fun main() {
    val acc = BankAccount(1000.0)
    acc.withdraw(350.0)
}
```

## Самопроверка и отладка (assert)

- **Функция assert():** проверяет внутренние инварианты алгоритма при отладке
- **Ключ JVM -ea:** включается при тестировании и не замедляет боевой сервер

## Предусловия в стандартной библиотеке

- **require(условие):** проверяет аргументы функции (бросает IllegalArgumentException)
- **check(условие):** проверяет состояние объекта (бросает IllegalStateException)
- **error("текст"):** явно обозначает недостижимую ветку или неверное состояние

# Пространства имён, пакеты и модули

## Пакеты (package) и импорт (import)

- **Пространства имён:** package задаёт логическую группу для классов и функций файла
- **Псевдонимы импорта:** import ... as позволяет переименовать класс при конфликте имён

## Модули и управление экспортом

- **Модуль (Module):** единица сборки (Gradle/Maven), объединяющая пакеты подпроекта
- **Экспорт наружу (public):** формирует открытый API для других модулей
- **Инкапсуляция (internal / private):** internal скрывает код внутри модуля, private — внутри файла

p2_07_packages_modules.kt

```kotlin
package app.analytics

import kotlin.math.PI as MATH_PI
import kotlin.math.roundToInt

// public — экспортируется внешним модулям
class MetricsReport(val name: String, val v: Double) {
    fun summary() = "$name: ${v.roundToInt()}"
}

// internal — доступен только внутри своего модуля
internal object InternalCalc {
    fun area(r: Double) = MATH_PI * r * r
}

// private — виден только внутри текущего файла
private fun log(msg: String) = "[LOG] $msg"
```

# Декларация, инстанцирование и границы

p2_08_generics_declaration.kt

```kotlin
// 1. Декларация обобщённого класса
class StorageBox<T>(private var item: T) {
    fun get(): T = item
    fun update(newItem: T) { item = newItem }
}

// 2. Ограничение типа (Upper Bound)
fun <T : Comparable<T>> findMax(a: T, b: T): T =
    if (a >= b) a else b

fun main() {
    val sBox = StorageBox<String>("Kotlin 2.4")
    val iBox = StorageBox(2026) // StorageBox<Int>
    println(findMax(42, 99))
    println(findMax("Alpha", "Omega"))
}
```

## Декларация и инстанцирование

- **Параметры типа <T>:** позволяют писать универсальные контейнеры и алгоритмы
- **Строгая типизация:** все проверки типов выполняются компилятором без ручных кастов
- **Вывод типа:** при вызове StorageBox(2026) компилятор сам подставляет <Int>

## Ограничения типов (Upper Bounds)

- **Верхняя граница:** запись <T : Comparable<T>> требует реализации интерфейса
- **Несколько условий:** ключевое слово where задаёт сразу несколько ограничений на T

# Использование: вариантность и reified

## Вариантность (out и in)

- **Ковариантность (out T):** тип только возвращается (Producer); сохраняет иерархию подтипов
- **Контравариантность (in T):** тип только принимается в методы (Consumer)

## Сохранение типа с inline + reified

- **Стирание типов JVM:** в обычных функциях параметр <T> стирается при компиляции
- **Модификатор reified:** в inline-функциях сохраняет тип T для проверок element is T

p2_09_generics_variance.kt

```kotlin
interface Producer<out T> { fun produce(): T }
interface Consumer<in T>  { fun consume(x: T) }

// Вещественный тип T благодаря inline + reified
inline fun <reified T> filterByType(list: List<Any>):
List<T> {
    val res = mutableListOf<T>()
    for (item in list) {
        if (item is T) res.add(item)
    }
    return res
}

fun main() {
    val mixed = listOf("Kotlin", 100, "JVM", 42)
    println(filterByType<String>(mixed)) // [Kotlin,
JVM]
}
```

КОЛЛЕКЦИИ

# Массивы, списки, множества и карты

p2_10_collections.kt

```kotlin
fun main() {
    val primes: IntArray = intArrayOf(2, 3, 5, 7)

    val readOnly: List<String> = listOf("Android",
"Web")
    val tasks = mutableListOf("Код", "Тесты")
    tasks.add("Релиз")

    val tags: Set<String> = setOf("jvm", "kotlin",
"jvm")
    // В множестве останется только 2 элемента

    val versions: Map<String, Double> = mapOf(
        "Kotlin" to 2.4,
        "Java" to 21.0
    )
    println(versions["Kotlin"])
}
```

## Архитектура коллекций Kotlin

- **Неизменяемые (Read-only):** List, Set, Map защищают данные от случайного изменения
- **Изменяемые (Mutable):** MutableList, MutableSet, MutableMap поддерживают добавление/удаление

## Основные структуры данных

- **IntArray / Array<T>:** непрерывный блок памяти; IntArray хранит примитивы без боксинга
- **List и Set:** List хранит элементы по индексу; Set гарантирует уникальность
- **Map<K, V>:** ассоциативный словарь пар ключ-значение ("Kotlin" to 2.4)

СТАНДАРТНАЯ БИБЛИОТЕКА ВВОДА-ВЫВОДА

# Работа с консолью и файловой системой

## Консольный ввод и вывод

- **Чтение строк:** readln() и безопасная функция readlnOrNull() без Scanner
- **Форматированный вывод:** println() с поддержкой строковых шаблонов "$user"

## Файловый ввод-вывод (kotlin.io)

- **Удобные расширения File:** методы writeText(), appendText(), readText(), readLines()
- **Автозакрытие ресурсов:** блоки use {} и useLines {} гарантированно закрывают поток файла

p2_11_io_files_console.kt

```kotlin
import java.io.File

fun main() {
    print("Введите имя: ")
    val user = readlnOrNull() ?: "Гость"

    val file = File("session_log.txt")
    file.writeText("Пользователь: $user\n")
    file.appendText("Статус: OK\n")

    // Безопасное построчное чтение с автозакрытием
    file.useLines { lines ->
        lines.forEach { println("Строка: $it") }
    }
    file.delete()
}
```

# Спасибо за внимание

готовы ответить на ваши вопросы